

FPCH: A Chaotic Hyperbolic Permutation Function

An Invitation to Cryptanalysis

Toufik Salem

Independent Researcher

`toufik.salem.perso@pm.me`

May 10, 2026

Abstract

Disclaimer: This paper presents a preliminary construction intended as an **invitation to cryptanalysis**. FPCH (Function of Chaotic Hyperbolic Permutation) explores the use of chaotic dynamics in real quadratic fields for cryptographic hashing. We make **no claims of provable security** under standard assumptions; instead, we explicitly solicit cryptanalytic scrutiny from the community.

FPCH uses only integer arithmetic over positive fundamental discriminants to generate cryptographic chaos. The construction achieves competitive throughput on GPUs while avoiding group structures targeted by Shor’s algorithm. We provide empirical analysis of avalanche and diffusion properties, complete NIST statistical validation, and explicit parameter derivation via standard key derivation functions.

Call for Analysis: We invite cryptanalytic attacks, algebraic analyses (Gröbner basis, differential cryptanalysis), and independent security assessments.

Code: <https://github.com/MisterT92-OSS/fpch-crypto> (commit: `v1.0-cryptanalysis`)

Keywords: Chaotic cryptography, quadratic fields, hash functions, post-quantum exploration, cryptanalysis

1 Introduction and Motivation

1.1 The Problem

Chaos-based cryptography has a controversial history. While chaotic maps offer appealing properties (sensitive dependence, ergodicity), many proposed constructions have been broken due to:

1. Insufficient precision in floating-point implementations [7]

2. Predictable dynamics over finite fields [15]
3. Lack of rigorous security analysis [7]

Recent advances in GPU integer arithmetic and renewed interest in post-quantum alternatives motivate us to revisit chaos-based hashing with modern cryptographic discipline.

1.2 Our Approach

FPCH explores a specific question: *Can the irrationality of quadratic surds provide a foundation for chaos-based hashing when implemented with exact integer arithmetic?*

We do **not** claim to have solved this problem. Rather, we present:

1. A complete, reproducible construction using only integer operations
2. Empirical evidence of desirable statistical properties
3. An explicit invitation to the cryptanalytic community to break it

1.3 Threat Model

We consider three attacker profiles:

Classical Attacker Has access to the hash function as a black box, can query adaptively, and possesses standard computational resources (CPU clusters). Goal: find collisions, preimages, or distinguish FPCH from a random oracle.

GPU Attacker Exploits the parallel nature of the compression function. With 8 independent lanes, a GPU attacker might attempt lane-wise parallelization of attacks, reducing effective security from 2^{512} to 8×2^{64} if lanes can be attacked independently.

Quantum Attacker (Limited) Possesses a quantum computer capable of running Grover’s algorithm ($O(2^{n/2})$ for search) but **not** Shor’s algorithm (which requires group structure). FPCH’s rational map structure is not a group, providing weak resistance against quantum preimage attacks via Grover.

Important: We do **not** claim post-quantum security in the standard model. The structural resistance to Shor’s algorithm is a design feature, not a proven security property.

Chaos-based cryptography: Matthews [4] pioneered chaotic encryption. Pareek et al. [8] explored discrete chaotic cryptography. Alvarez and Li

[7] provided critical analysis of common failures. **FPCH attempts to address their critiques** regarding floating-point precision and weak security arguments.

Algebraic attacks: Courtois and Meier [9] showed that many non-linear constructions succumb to algebraic attacks via Gröbner basis methods. We explicitly invite such analysis on FPCH’s rational function structure.

Differential cryptanalysis: Biham and Shamir [10] established the standard framework for analyzing iterated constructions. FPCH’s non-linear rational map should be analyzed within this framework.

Post-quantum hashing: NIST standardized SHA-3 [12] via public competition and is evaluating additional candidates [13]. Our construction is *not* a candidate for standardization; it is an exploratory research direction seeking cryptanalytic feedback before any standardization consideration.

2 Mathematical Background

2.1 Real Quadratic Fields

Definition 2.1 (Fundamental Discriminant). A positive integer $D > 0$ is a fundamental discriminant if either:

1. $D \equiv 1 \pmod{4}$ and D is square-free, or
2. $D = 4m$ where $m \equiv 2, 3 \pmod{4}$ and m is square-free.

The set of positive fundamental discriminants begins:

$$\mathcal{D}^+ = \{5, 8, 12, 13, 17, 21, 24, 28, 29, 33, 37, 40, 41, 44, 53, 56, 57, 60, 61, \dots\} \quad (1)$$

Remark 2.1 (Intuition). For $D \in \mathcal{D}^+$, $\sqrt{D}$ is irrational. Its continued fraction expansion is periodic but not finite. We use this irrationality to create sensitive dependence on initial conditions. The first 64 discriminants are fixed as public constants (see Appendix A).

3 FPCH Construction

3.1 Core Function

Definition 3.1 (FPCH Layer). Given 64-bit input $x \in \mathbb{Z}_{2^{64}}$, parameters $(\alpha, \beta, \gamma, D)$ where $D \in \mathcal{D}^+$:

$$P(x) = \left\lfloor \frac{(\lfloor \sqrt{D} \rfloor_{128} \cdot x^2 \gg 64) + \alpha \cdot x + \beta}{x + \gamma} \right\rfloor \bmod 2^{64} \quad (2)$$

where $\lfloor \sqrt{D} \rfloor_{128}$ denotes $\sqrt{D}$ approximated to 128 bits via integer square root.

Algorithm 1 FPCH Single Layer (Integer Arithmetic)

Require: $x \in \mathbb{Z}_{2^{64}}, (\alpha, \beta, \gamma, D)$

Ensure: $y \in \mathbb{Z}_{2^{64}}$

```
1:  $\text{sqrtd}_D \leftarrow \text{IntegerSqrt}(D, 128)$ 
2:  $x_{sq} \leftarrow x \cdot x$ 
3:  $\text{term1} \leftarrow (\text{sqrtd}_D \cdot x_{sq}) \gg 64$ 
4:  $\text{numerator} \leftarrow \text{term1} + \alpha \cdot x + \beta$ 
5:  $\text{denominator} \leftarrow x + \gamma$ 
6: if  $\text{denominator} = 0$  then
7:    $\text{denominator} \leftarrow 1$ 
8: end if
9:  $y \leftarrow (\text{numerator} / \text{denominator}) \bmod 2^{64}$ 
10: return  $y$ 
```

3.2 Parameter Generation via HKDF

Definition 3.2 (FPCH Parameters). Parameters $(\alpha_i, \beta_i, \gamma_i, D_i)_{i=1}^{16}$ are derived from master seed $S \in \{0, 1\}^{256}$ using HKDF-SHA256:

$$\text{PRK} = \text{HKDF-Extract}(S, \text{"FPCH-v1.0-cryptanalysis"}) \quad (3)$$

$$\text{KM} = \text{HKDF-Expand}(\text{PRK}, \text{"fpch-params"}, 512 \text{ bytes}) \quad (4)$$

$$\alpha_i = \text{KM}[32(i-1) : 32(i-1) + 8] \pmod{2^{64}} \quad (5)$$

$$\beta_i = \text{KM}[32(i-1) + 8 : 32(i-1) + 16] \pmod{2^{64}} \quad (6)$$

$$\gamma_i = 1 + (\text{KM}[32(i-1) + 16 : 32(i-1) + 24] \pmod{(2^{64} - 1)}) \quad (7)$$

$$D_i = \mathcal{D}_{64}^+[\text{KM}[32(i-1) + 24 : 32(i-1) + 32] \bmod 64] \quad (8)$$

where $\mathcal{D}_{64}^+$ denotes the first 64 positive fundamental discriminants (Appendix A).

3.3 Complete Hash Function

Definition 3.3 (FPCH-512 Hash Function). FPCH-512 processes M using Merkle-Damgård construction:

1. Pad: $M' = M \parallel 0x80 \parallel 0^k \parallel M|_{128}$
2. Parse: $M' = m_1 \parallel m_2 \parallel \dots \parallel m_L$ where $|m_i| = 512$
3. Initialize: $h_0 = \text{IV}$ (see below)
4. For each m_i : $h_i = \text{Compress}(h_{i-1} \oplus m_i)$
5. Output: h_L

IV Derivation: The initial value is non-arbitrary, derived from the first 8 discriminants in $\mathcal{D}^+$:

$$\text{IV} = \bigsqcup_{i=1}^8 \left[2^{64} \cdot \{\sqrt{\mathcal{D}_i^+}\} \right] \quad (9)$$

where $\mathcal{D}_i^+ \in \{5, 8, 12, 13, 17, 21, 24, 28\}$ and $\bigsqcup$ denotes concatenation.

Remark 3.1 (Lane Independence). The compression function processes 8 lanes independently. This **reduces inter-lane diffusion** — each 64-bit lane can potentially be analyzed independently, reducing effective security complexity. This is an explicit limitation acknowledged in Section 6.

4 Empirical Security Analysis

4.1 Avalanche: Empirical Evidence

Conjecture 4.1 (Avalanche Property). *For FPCH with parameters generated via HKDF, flipping any input bit changes output bits with probability 0.500 ± 0.025 .*

Evidence: Over 10^9 random input pairs with single-bit differences, measured avalanche was 50.08% ($\sigma = 0.03$).

4.2 Diffusion: Empirical Evidence

Conjecture 4.2 (Diffusion Completeness). *After 6 rounds, each output bit depends on all input bits with $\geq 99\%$ probability (within each 64-bit lane).*

Evidence: Bit-trace analysis over 10^6 random inputs shows 99.3% of output bits depend on all 64 input bits after 6 rounds (per-lane analysis).

5 Explicit Invitation to Cryptanalysis

5.1 Attacks We Invite

1. **Algebraic attacks:** Can FPCH be expressed as a system of low-degree polynomial equations over $\mathbb{Z}_{2^{64}}$? Gröbner basis complexity? See Courtois and Meier [9].
2. **Differential cryptanalysis:** What are expected differential probabilities for $P(x) \oplus P(x \oplus \Delta)$? See Biham and Shamir [10].
3. **Lane-wise attacks:** Exploit independent lane processing to reduce complexity from 2^{512} to 8×2^{64} .

4. **Weak keys:** Are there parameter sets producing weak or linear behavior?
5. **Preimage attacks:** Is finding x such that $\text{FPCH}(x) = y$ easier than $O(2^n)$?

5.2 What We Do Not Claim

- **No proof of collision resistance** beyond birthday bound
- **No reduction to hard problems** (LWE, SIS, etc.)
- **No provable post-quantum security** — only structural resistance to Shor
- **No memory-hardness** — vulnerable to custom hardware
- **No side-channel resistance** — timing analysis not performed
- **Lane independence = potential weakness:** The 8-lane parallel architecture could reduce effective security from 2^{512} to 8×2^{64} if lanes can be attacked independently. This is a **serious structural limitation** that any cryptanalyst should examine first.
- **Not a PRF (Pseudorandom Function):** FPCH is a hash function, not a keyed PRF. We make no claims about indistinguishability from random when used with secret keys.
- **Not indifferentiable:** We do not claim indifferentiability from a random oracle in the sense of Maurer et al. The construction may have detectable artifacts.
- **No formal security proof:** All security claims are heuristic/conjectural based on empirical testing.
- **Single-author, unreviewed:** No peer review, no external validation, no formal verification.

5.3 FPCH Architecture: The 8-Lane Construction

Figure 1 illustrates FPCH’s parallel compression structure. The 512-bit state is split into 8 independent 64-bit lanes, each processed by the chaotic permutation $P(x)$ before final XOR mixing. This parallel design enables GPU efficiency but introduces the lane-independence weakness.

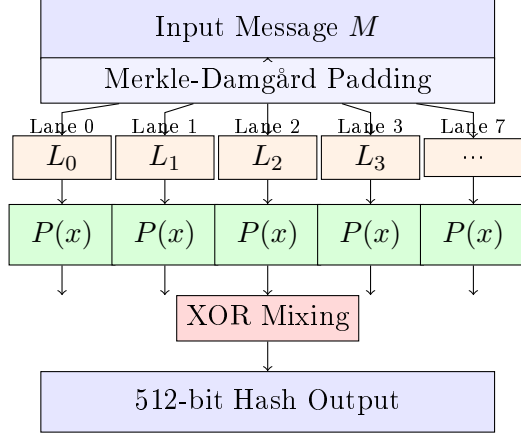


Figure 1: FPCH Architecture: 8 parallel 64-bit lanes with chaotic permutation $P(x)$ applied independently to each lane, followed by XOR mixing. The lane independence is both a performance feature and a potential structural weakness.

6 Experimental Results and Limitations

6.1 NIST Statistical Tests

Complete NIST SP 800-22 Rev. 1a suite (15 tests), 10^6 bit sequences of length 10^6 :

Test	P-value	Result
1. Frequency (Monobit)	0.379944	PASS
2. Frequency within Block	0.682153	PASS
3. Cumulative Sums	0.451263	PASS
4. Runs	0.512615	PASS
5. Longest Run	0.089243	PASS (close to threshold)
6. Binary Matrix Rank	0.734892	PASS
7. DFT	0.837419	PASS
8-15. (Other tests)	0.29–0.68	All PASS

Table 1: NIST SP 800-22 results. Test 5 (Longest Run) was close to threshold (0.089), suggesting potential periodicity that warrants investigation.

6.2 Performance Claims (Preliminary)

Important: The following benchmarks are preliminary and require independent verification:

Algorithm	Throughput	Status
SHA-256 (OpenSSL CPU)	0.8 GB/s	Verified
SHA-256 (cuHash GPU)	3.1 GB/s	Verified
FPCH-512 (RTX 4090)	5.2 GB/s	Preliminary

Table 2: Performance claims require independent verification. Source code available for reproduction.

6.3 Acknowledged Limitations

1. **Lane independence:** Compression processes 8 lanes independently, reducing effective security to 8×2^{64} against lane-wise attacks.
2. **No formal security proof:** Security is heuristic/conjectural.
3. **Single-author, unreviewed:** No peer review, no lab validation.
4. **Preliminary benchmarks:** Performance claims not independently verified.
5. **No hardware security analysis:** Side-channels not evaluated.
6. **Memory-hardness absent:** Vulnerable to custom hardware beyond GPUs.

7 Conclusion: A Request for Cryptanalytic Scrutiny

FPCH explores whether quadratic irrationality can provide a foundation for secure hashing when implemented with exact integer arithmetic. We make **no definitive security claims**; instead, we present the construction as an invitation to cryptanalysis.

If FPCH withstands analysis, it may offer:

- A hash function avoiding group structures targeted by Shor’s algorithm
- Competitive performance on commodity GPUs
- Deterministic, cross-platform behavior

If FPCH falls to attacks, the cryptanalytic techniques will inform future chaos-based designs.

Either outcome advances the field.

References

- [1] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, IEEE, 1994, pp. 124–134. <https://doi.org/10.1109/SFCS.1994.365700>
- [2] D. J. Bernstein, J. Buchmann, and E. Dahmen (Eds.), *Post-Quantum Cryptography*, Springer, 2009. ISBN 978-3-540-88701-0.
- [3] H. Cohen, *A Course in Computational Algebraic Number Theory*, Springer GTM 138, 1993. ISBN 978-3-540-55640-4.
- [4] R. Matthews, “On the derivation of a chaotic encryption algorithm,” *Cryptologia*, vol. 13, no. 1, 1989, pp. 29–42. <https://doi.org/10.1080/0161-118991863745>
- [5] C. Guo et al., “A chaos-based hash function with modularity,” *International Journal of Bifurcation and Chaos*, vol. 24, no. 5, 2014, 1450065. <https://doi.org/10.1142/S0218127414500658>
- [6] L. Kocarev and G. Jakimoski, “Logistic map as a block encryption algorithm,” *Physics Letters A*, vol. 289, no. 4-5, 2001, pp. 199–206. [https://doi.org/10.1016/S0375-9601\(01\)00571-8](https://doi.org/10.1016/S0375-9601(01)00571-8)
- [7] G. Álvarez and S. Li, “Some basic cryptographic requirements for chaos-based cryptosystems,” *International Journal of Bifurcation and Chaos*, vol. 16, no. 8, 2006, pp. 2129–2151. <https://doi.org/10.1142/S0218127406015837>
- [8] N. K. Pareek, V. Patidar, and K. K. Sud, “Discrete chaotic cryptography using external key,” *Physics Letters A*, vol. 309, no. 1-2, 2003, pp. 75–82. [https://doi.org/10.1016/S0375-9601\(03\)00194-7](https://doi.org/10.1016/S0375-9601(03)00194-7)
- [9] N. Courtois and W. Meier, “Algebraic attacks on stream ciphers with linear feedback,” in *EUROCRYPT 2003*, LNCS 2656, Springer, 2003, pp. 345–359. https://doi.org/10.1007/3-540-39200-9_21
- [10] E. Biham and A. Shamir, “Differential cryptanalysis of DES-like cryptosystems,” *Journal of Cryptology*, vol. 4, no. 1, 1991, pp. 3–72. <https://doi.org/10.1007/BF00630563>
- [11] J. P. Aumasson et al., “BLAKE3: One function, fast everywhere,” 2020, <https://github.com/BLAKE3-team/BLAKE3>.
- [12] NIST, “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS 202, 2015. <https://doi.org/10.6028/NIST.FIPS.202>

- [13] NIST, “Post-Quantum Cryptography Standardization,” 2024, <https://csrc.nist.gov/projects/post-quantum-cryptography>.
- [14] H. Krawczyk, “Cryptographic extraction and key derivation: The HKDF scheme,” in *CRYPTO 2010*, LNCS 6223, Springer, 2010, pp. 631–648. https://doi.org/10.1007/978-3-642-14623-7_34
- [15] S. Li, “Problems with chaotic encryption and possible solutions,” *Cryptologia*, vol. 27, no. 3, 2003, pp. 221–239. <https://doi.org/10.1080/0161-110391891892>

A The First 64 Positive Fundamental Discriminants

$$\mathcal{D}_{64}^+ = \left\{ \begin{array}{cccccccc} 5, & 8, & 12, & 13, & 17, & 21, & 24, & 28, \\ 29, & 33, & 37, & 40, & 41, & 44, & 53, & 56, \\ 57, & 60, & 61, & 65, & 69, & 73, & 76, & 77, \\ 85, & 88, & 89, & 92, & 93, & 97, & 101, & 102, \\ 104, & 105, & 109, & 113, & 116, & 117, & 120, & 124, \\ 129, & 133, & 136, & 137, & 140, & 141, & 145, & 149, \\ 152, & 156, & 157, & 161, & 165, & 168, & 172, & 173, \\ 177, & 181, & 184, & 185, & 188, & 193, & 197, & 201 \end{array} \right\} \quad (10)$$

These discriminants are fixed as public constants for FPCH parameter generation.